



Cofinancé par
l'Union européenne



TP6 CDA PRF · RNCP 37873 — CONCEPTEUR
DÉVELOPPEUR D'APPLICATIONS (TP6 CDA)

COURS HTML5, CSS3 & BOOTSTRAP

HTML/CSS/Bootstrap

J2 — CSS avancé

PROMO 2026-D03, CDA JS FULLSTACK IA

Formateur : Robin Hotton, rhotton@diginamic.fr

Formateur Robin HOTTON

Promotion 2026-D03

Version 1.2 — 2026

Agenda du jour

Matin — CSS puissant

- IX** — Sélecteurs avancés
- X** — Cascade & spécificité
- XI** — Flexbox + Froggy

Après-midi — Layout + responsive

- XII** — Grid + Garden
- XIII** — Responsive mobile-first
-  **Bloc DigiShop responsive**

À la fin de la journée, vous saurez :

- ✓ Cibler n'importe quel élément avec le bon sélecteur
- ✓ Résoudre un conflit CSS sans **!important**
- ✓ Mettre en page avec Flexbox (1D) et Grid (2D)
- ✓ Adapter un site du mobile au desktop

IX — Sélecteurs avancés

Cibler précisément sans surcharger le HTML

D'après vous...

Q1 — `nav ul li` (avec espaces) vs `nav > ul > li` (avec `>`) : quelle différence ?

◆ DESCENDANT VS ENFANT DIRECT

`nav ul li` — **tous les** `<li>` **descendants** (même sous-menus)

`nav > ul > li` — **uniquement les enfants directs**

Q2 — `:hover` et `::before` : quelle différence ?

❗ PSEUDO-CLASSE VS PSEUDO-ÉLÉMENT

`:hover` (un `:`) — un **état**

`::before` (deux `::`) — un **élément virtuel** ajouté par CSS

Q3 — Ajouter une icône ↗ après tous les liens externes **sans toucher au HTML** ?

◆ SÉLECTEUR D'ATTRIBUT + PSEUDO-ÉLÉMENT

```
a[target="_blank"]::after { content: " ↗"; color: gray; }
```

📖 COMBINAIREUR = relation entre deux
sélecteurs (descendant, `>` enfant, `+`
frère immédiat, `~` frère général)

5 familles de sélecteurs

Simple

`h1` · `.classe` · `#id`

Type (élément HTML), classe (réutilisable), id (unique sur la page).

Combinateurs

`A B` (descendant) · `A > B` (enfant) · `A + B` (frère direct) ·
`A ~ B` (frères)

Relient deux sélecteurs pour cibler par hiérarchie DOM.

Attributs

`[type="email"]` · `[href^="https://"]` ·
`[class*="card"]`

Match exact (`=`), début (`^=`), contient (`*=`), fin (`$=`).

Pseudo-classes (`.:`) + pseudo-éléments (`.::`)

`:hover` `:focus` `:nth-child(odd)` `:not(.active)`
`::before` `::after` `::placeholder`

État dynamique (`:`) vs élément virtuel généré par CSS (`::`).

Code — sélecteurs en action

```
/* Lien externe : icône ajoutée par CSS */
a[target="_blank"]::after {
  content: " ↗";
  color: gray;
}

/* Une ligne sur deux dans un tableau */
tbody tr:nth-child(even) {
  background: #f5f5f5;
}

/* Champ email invalide */
input[type="email"]:invalid {
  border-color: red;
}

/* Premier paragraphe d'un article */
article > p:first-of-type {
  font-size: 1.2rem;
  font-weight: 500;
}
```

Exercice — Sélecteurs CSS

DÉFIS ● · **CSS DINER** · 15-30 MIN SELON NIVEAU

32 niveaux gamifiés pour s'entraîner sans installer quoi que ce soit — uniquement le navigateur.

N. 1-15 — sélecteurs de base, descendant, classes, id

N. 16-25 — pseudo-classes `:nth-child` , `:first-of-type` , combinateurs

N. 26-32 — sélecteurs d'attributs `[...^=]` , `[...*=]` , combinaisons avancées

À chaque niveau : une assiette d'éléments HTML à cibler avec le bon sélecteur. Feedback immédiat, indices disponibles si on bloque.

Correction : andersjensen.org/solutions-to-css-diner

X — Cascade & spécificité

Comprendre qui gagne quand deux règles s'opposent

D'après vous...

Q1 — Vous écrivez `p { color: red }` puis `.intro { color: blue }`. Sur `<p class="intro">`, quelle couleur s'affiche ?

◆ BLEU — LA CLASSE L'EMPORTE

Classe (poids 10) bat toujours **type** (poids 1). L'ordre joue **qu'à spécificité égale**.

Q2 — Spécificité de `#header .menu a:hover` ?

❶ (1, 2, 1)

`1 id · 2 classes/pseudo-classes · 1 type` → format `(a, b, c)`.

Q3 — `!important` pour forcer une couleur — pourquoi c'est un signal d'alarme ?

⚠ GUERRE D'ESCALADE

Court-circuite la cascade → l'auteur suivant doit mettre `!important` partout pour corriger. À réserver aux overrides ponctuels.

Les 3 critères de la cascade

✦ 1. IMPORTANCE

`!important` > règle normale

Réservé aux overrides critiques.

ℹ 2. SPÉCIFICITÉ

`#id` (100) > `.classe` (10) > `type` (1)

Format **a, b, c**.

◆ 3. ORDRE

À spécificité égale, la **dernière** règle écrite gagne.

✎ VÉRIFIER EN DEVTOOLS

Panneau **Styles** → les règles **barrées** sont celles écrasées par la cascade. Outil pédago imbattable pour comprendre un conflit.

Code — héritage et `inherit`

```
/* Le lien hérite de la couleur du parent
   au lieu du bleu UA par défaut */
.navbar {
  color: white;
}

.navbar a {
  color: inherit;      /* reprend le white du parent */
  text-decoration: none;
}
```

⚠ POURQUOI `INHERIT` EST NÉCESSAIRE ICI

Les liens n'héritent **pas** naturellement de `color` — la **UA stylesheet** impose son bleu. `inherit` court-circuite ce comportement.

Exercice — Sélecteurs, spécificité & cascade

EXERCICE `EX/10\_CASCADE-CONFLITS.MD` • ● + ● • 25 MIN

Diagnostiquer à la main un conflit CSS — la cascade n'est pas magique, elle se calcule.

● Mémoriser le tableau de spécificité (id × 100, classe × 10, type × 1) + observer un cas live : `a` vs `.link-cta` vs `#content a` — qui gagne ?

● Calculer la spécificité de 8 sélecteurs (`nav > ul > li a:hover` , `#app .sidebar .menu a` ...) puis débbugger un `bug.html` où `!important` fait la loi. **Refactor** : titre vert (`#0a8a4a`), zéro `!important` , spécificité minimale.

Indice : DevTools panneau Styles barre les règles écrasées — c'est le verdict visuel de la cascade.

XI — Flexbox

Mise en page 1D — la base 2026

D'après vous...

Q1 — Comment **centrer parfaitement** un élément en horizontal ET vertical dans son parent ?

◆ TROIS LIGNES, FINI LE CAUCHEMAR

```
.parent {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

Q2 — Navbar : pousser le menu à droite, logo à gauche ?

❗ MARGIN-LEFT: AUTO SUR LE MENU

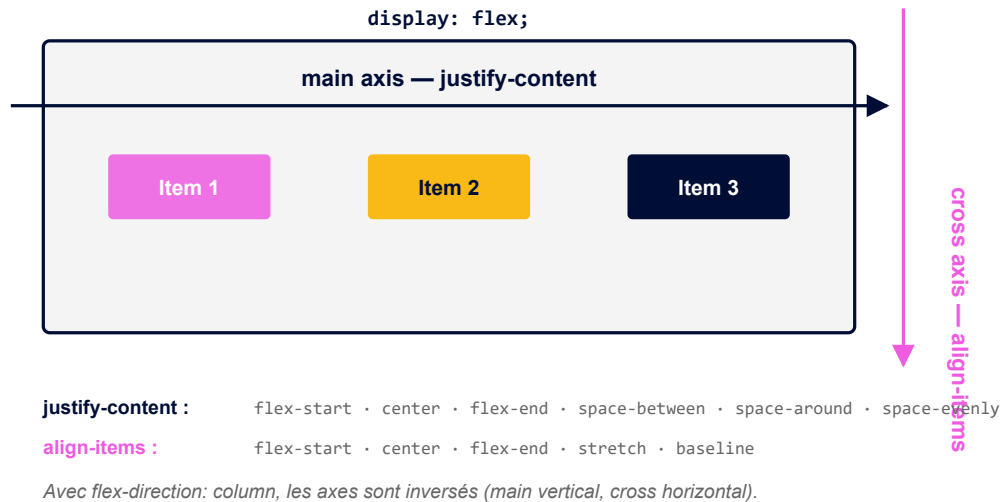
`flex` sur la nav + `margin-left: auto` sur le `<ul>` → menu poussé à droite.

Q3 — `gap: 1rem` sur un Flexbox : que fait-il ?

◆ ESPACE PROPRE ENTRE ITEMS

`gap` ajoute **1rem entre chaque flex item** — fini `margin` + `:last-child { margin: 0 }`.

Flexbox — 2 axes, 4 propriétés clés



Conteneur

`display: flex`
`gap` — espace entre items

Axe principal — `justify-content`

`flex-start` · `center` · `space-between` · `space-around`

Axe perpendiculaire — `align-items`

`stretch` (défaut) · `center` · `flex-start` · `flex-end`

Code — navbar Flexbox classique

```
<nav class="navbar">
  <a class="logo">Mon Site</a>
  <ul class="menu">
    <li><a href="#">Accueil</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>
```

```
.navbar {
  display: flex;
  align-items: center;
  padding: 1rem;
}

.menu {
  display: flex;
  gap: 1.5rem;
  margin-left: auto; /* pousse le menu à droite (Q2) */
  list-style: none;
}
```


Exercice — Flexbox Froggy

EXERCICE ● · **FLEXBOXFROGGY.COM** · 30 MIN EN SÉANCE

24 niveaux gamifiés pour placer des grenouilles sur des nénuphars — directement dans le navigateur, sans installation.

N. 1-8 — `justify-content` , `align-items`

N. 9-16 — `flex-direction` , `flex-wrap`

N. 17-24 — `order` , `align-self` , combinaisons

Une grenouille = un `flex item` , le nénuphar = la position cible. Feedback immédiat à chaque tentative.

XII — CSS Grid

Mise en page 2D — puissant et magique

D'après vous...

Q1 — Une galerie qui s'adapte automatiquement : 4 colonnes desktop, 3 tablette, 2 mobile, **sans une seule media query**. Possible ?

◆ UNE LIGNE MAGIQUE

```
.galerie {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
  gap: 1.5rem;  
}
```

Le navigateur calcule combien de colonnes tiennent (min 250px, partage **1fr**).

Q2 — Dashboard header/sidebar/main/footer : Flexbox ou Grid ?

❗ GRID — C'EST DU 2D

2D (lignes ET colonnes) → **Grid** avec **grid-template-areas** . Flex pour 1D, Grid pour le layout — **souvent combinés**.

Q3 — **auto-fit** vs **auto-fill** dans **repeat(...)** : quelle différence ?

Grid vs Flexbox — quand quoi ?

Flexbox — 1D

Une **ligne** OU une **colonne**.

Navbar

Barre de boutons

Liste verticale

Composants internes

Grid — 2D

Lignes **ET** colonnes en même temps.

Layout de page complet

Dashboard

Galerie responsive

Mise en page magazine

♦ COMBINAISON GAGNANTE

Grid pour le layout de page (zones header/sidebar/main/footer) · **Flexbox** dans chaque zone interne (alignement des items).

Code — Grid layout dashboard

```
.dashboard {  
  display: grid;  
  grid-template-areas:  
    "header header header"  
    "sidebar main main"  
    "footer footer footer";  
  grid-template-columns: 200px 1fr 1fr;  
  grid-template-rows: auto 1fr auto;  
  gap: 1rem;  
  min-height: 100vh;  
}  
  
.dashboard > header { grid-area: header; }  
.dashboard > nav    { grid-area: sidebar; }  
.dashboard > main    { grid-area: main; }  
.dashboard > footer  { grid-area: footer; }
```

Exercice — CSS Grid Garden

EXERCICE ● · **CSSGRIDGARDEN.COM** · 30 MIN EN SÉANCE

28 niveaux gamifiés pour arroser ses carottes et chasser les mauvaises herbes — manipuler une grille 2D dans le navigateur.

N. 1-10 — `grid-column-start/end` , `repeat()`

N. 11-20 — `grid-template-columns/rows` , `fr`

N. 21-28 — `grid-template-areas` , `auto-fit`

Plus exigeant que Flexbox Froggy : la 2D demande plus de visualisation mentale. Ne pas viser le 100 %.



XIII — Responsive mobile-first

Adapter aux écrans et accessibilité

D'après vous...

Q1 — Vous oubliez `<meta name="viewport">` dans le `<head>` d'un site mobile. Que se passe-t-il ?

⚠ ZOOM-OUT ~980PX, MEDIA QUERIES INOPÉRANTES

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Sans cette balise → texte minuscule, design cassé, `@media (max-width...)` muettes.

Q2 — `min-width` ou `max-width` : quel sens en 2026 ?

◆ MOBILE-FIRST — `MIN-WIDTH`

CSS mobile d'abord, règles desktop **ajoutées** via `@media (min-width: ...)`. Standard 2026 (Bootstrap, Tailwind).

Q3 — Désactiver les animations pour un utilisateur sensible aux mouvements ?

Breakpoints courants

Échelle pragmatique

0 - 575px mobile portrait
576 - 767px mobile paysage
768 - 991px tablette
992 - 1199px desktop
1200px+ grand desktop

Règle 2026

2 à 3 breakpoints suffisent dans 90 % des projets.

Les ajouter **au besoin**, pas par anticipation. Bootstrap utilise sm/md/lg/xl/xxl, mais c'est un framework.

Code — mobile-first

```
/* Base : mobile (≤ 767px) – règles écrites en premier */
.carte {
  width: 100%;
  padding: 1rem;
}

/* Tablette */
@media (min-width: 768px) {
  .carte {
    width: 50%;
  }
}

/* Desktop */
@media (min-width: 1200px) {
  .carte {
    width: 33.33%;
  }
}
```

Exercice — Responsive sur DigiShop

EXERCICE `EX/13\_RESPONSIVE-MOBILE-FIRST.MD` • ● 10 MIN + ● 20 MIN

Rendre le **DigiShop déjà construit** lisible et utilisable du mobile 320px au desktop large — approche **mobile-first** stricte.

Header en colonne sur < 768px

Grille produits : 3 col desktop → 1 col mobile

Texte lisible sur 320px (iPhone SE)

`prefers-reduced-motion` actif sur toutes les animations

Test : DevTools mode responsive, slider de 320px à 1440px — aucun overflow horizontal, rien qui se chevauche.



● **Récap J2 — DigiShop responsive**

Mini-TP autonomie guidée — réinvestir tous les concepts du jour

● Récap J2 — Mission

FORMATIF · 95 MIN EN SÉANCE · AUTONOMIE GUIDÉE — `EX/DIGISHOP/02\_RESPONSIVE.MD`

Faire passer DigiShop d'une mise en page fixe à un site **pleinement responsive** en 5 étapes :

1. Sélecteurs ciblés + cascade clean (spécificité max 0,2,1, zéro **!important**)
2. Header en **Flexbox** (logo gauche + nav droite)
3. Grille de produits en **CSS Grid** (**auto-fit** + **minmax**)
4. Responsive mobile-first sur **3 breakpoints** (375 / 768 / 1024)
5. Validation 3 viewports + Lighthouse Accessibility ≥ 95

◆ DÉFIS ●

Mode sombre via **prefers-color-scheme: dark**

Container queries (**@container**) sur une card produit

Animations **prefers-reduced-motion**

Conclusion J2

À retenir + cap sur J3

À retenir — J2

Cibler

`A B` descendant vs `A > B` enfant direct

Spécificité = (ids, classes, types)

DevTools panneau Styles pour debug

Jamais d' `!important`

Mettre en page

Flexbox = 1D (composants)

Grid = 2D (layout)

Mobile-first + `min-width`

`prefers-reduced-motion` en a11y

◆ LE SNIPPET À MÉMORISER — GALERIE RESPONSIVE SANS MEDIA QUERY

```
display: grid;  
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
gap: 1rem;
```

Demain — J3 : Bootstrap + TP final

Matin — Bootstrap

Intro & 3 piliers — grille, utilities, composants

Grille 12 col + utilities — gain de temps réel

Composants — navbar, cards, modals

● **Bloc DigiShop Bootstrap**

Après-midi — Choix + TP

Personnalisation — variables CSS, mode sombre

Custom vs Bootstrap vs Tailwind — décider

🏆 **TP sommatif /20** — Atelier Céramique

📄 **QCM final** (20 Q sur les 3 jours)



TP6 CDA PRF · RNCP 37873 — CONCEPTEUR
DÉVELOPPEUR D'APPLICATIONS (TP6 CDA)

COURS HTML5, CSS3 & BOOTSTRAP

Merci !

À DEMAIN POUR J3 — BOOTSTRAP + TP SOMMATIF

Questions, retours, demandes de rattrapage — c'est le moment.

Robin Hotton · rhotton@diginamic.fr

Formateur Robin HOTTON

Promotion 2026-D03

Version 1.2 — 2026